

• Decision Trees Geometric Intuition

* We have studied a lot of classification and regression techniques so far.

1. KNN is simple and heuristic based method: we take each datapoint and given an instance of points at runtime we take all the points to make a decision.

2. Naive Bayes is a probabilistic ^{Bayesian} method

3. Logistic Regression and Linear Regression both are also probabilistic but they are also inspired by Geometric Intuition and have hyperplanes.

SVM on the other hand is based on kernel trick which makes it special while being probabilistic and geometric along with hyperplanes.

All the three methods can be used for classification and Regression.

* Decision Trees are simply nested if-else based classifier and they have a program-matic interpretation.

Let's say we have the Iris dataset, where the features are $X_i = \langle SL, PL, SW, PW \rangle$ and we have three classes $\rightarrow$ Setosa, Versicolour and Virginica.

If our sample classifier looks as below:-

if $PL < a$:

$y_i = 1$

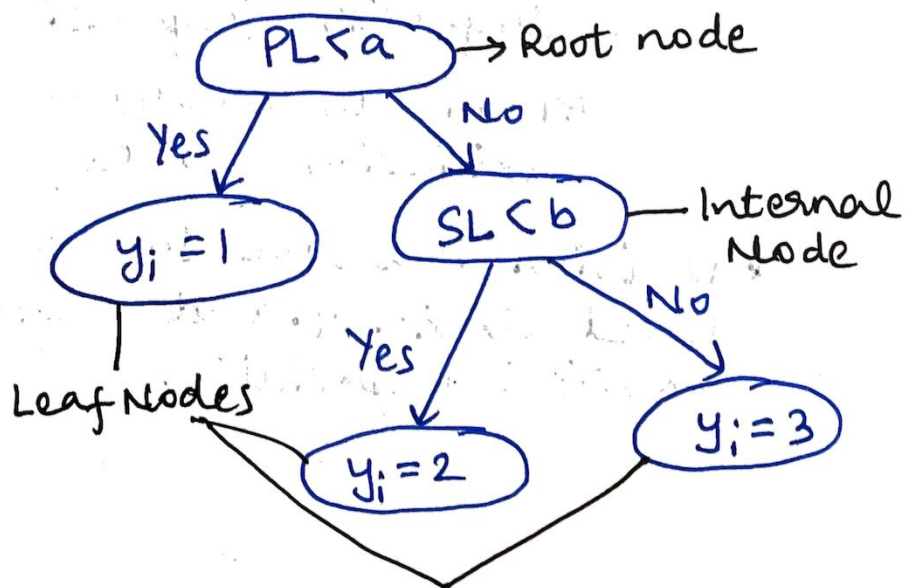
else:

if $SL < b$:

$y_i = 2$

else: $y_i = 3$

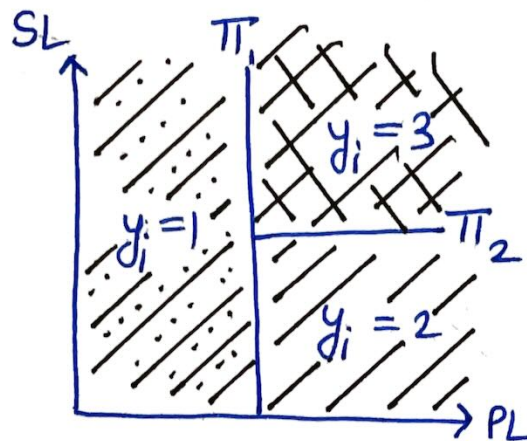
We can build a Decision Tree from above nested if-else classifier:-



NOTE: All leaf nodes are decision making.

Geometric Intuition

Suppose, we have two axes, SL and PL ,



A decision tree is always a set of axes parallel hyperplanes that divide the entire region into hypercubes and hypercuboids.

Properties of Entropy:

* For 2 class, Entropy lies between $[0, 1]$.

Case 1

Given a dataset D , let $Y^+ = 99\%$ and $Y^- = 1\%$.
then, entropy seems to decrease, ie, $H(Y) = 0.0801$.

Case 2

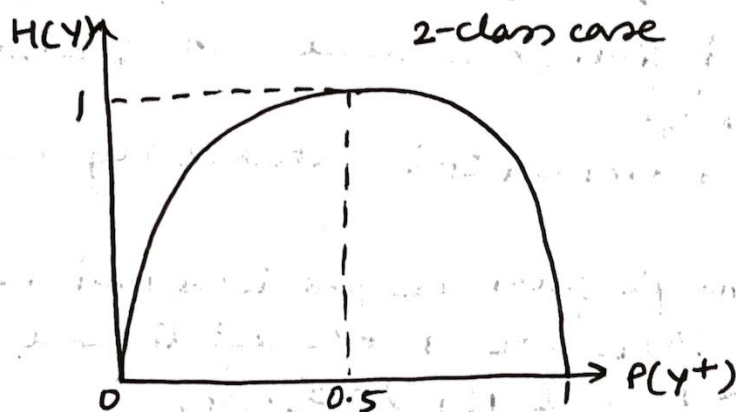
Given a dataset D , let $Y^+ = 50\%$ and $Y^- = 50\%$.
then, entropy seems maximum, ie, $H(Y) = 1.0$

Case 3

Given a dataset D , let $Y^+ = 0\%$ and $Y^- = 100\%$.
then, entropy seems minimum, ie, $H(Y) = 0.0$

Here, one class fully dominates, hence entropy is minimum.

Fig:



35.3

• Decision Tree: Entropy

Entropy is used in case of building a Decision Tree after training is done. Entropy is introduced in Information Theory.

Suppose Y is a random variable that can take K values $Y_1, Y_2, Y_3, Y_4, \dots, Y_K$

Then Entropy of Y is denoted as $H(Y)$.

$$H(Y) = - \sum_{i=1}^K P(Y_i) \log_b(P(Y_i))$$

Here b is base for log value. K is category of class labels. If it is a 2-class classification problem, then $K=2$, which is Binary Classification. So b would also be 2. So $\log$ would be written as 'lg'.

If $b=e$ (euler no.) then it is natural log where $e=2.718$

Let's take example from the previous topic. Suppose Y is "Play Tennis". There are 9 +ve points and 5 -ve points. Then $P(Y^+) = \frac{9}{14}$ and $P(Y^-) = \frac{5}{14}$. So,

$$H(Y^+) = - \left[\left(\frac{9}{14} \right) \lg \left(\frac{9}{14} \right) + \left(\frac{5}{14} \right) \lg \left(\frac{5}{14} \right) \right] = 0.94$$

$$\downarrow$$

$$\frac{\# \text{ of +ve pts}}{\text{Total pts}}$$

$$\downarrow$$

$$\% \text{ of +ve pts}$$

$$\downarrow$$

$$\frac{\# \text{ of -ve pts}}{\text{Total pts}}$$

$$\downarrow$$

$$\% \text{ of -ve pts}$$

- Decision Trees

- * Decision Trees are a non-parametric supervised learning method used for classification and regression.

- * The goal is to create a model that predicts the value of a target variable by learning simple decision rules inferred from data features.

- * A tree can be seen as a piecewise constant approximation.

- * For instance, in the example below, decision trees learn from data to approximate a sine curve with a set of if-then-else decision rules. [Fig on DT Page on SKlearn]

- * The deeper the tree, the more the complex the decision rules and the fitter the model.

- Advantages of Decision Trees

- * Simple to understand and to interpret. Trees can be visualized.

- * Requires little data interpretation. Other techniques often require data normalization, dummy variables need to be created and blank values to be removed.

- * Note: however, this module does not support missing values.

- * The cost of using the tree (ie, predicting the data) is logarithmic in the no. of datapoints used to train the tree.
- * Able to handle both numerical and categorical data. However, Scikit-learn implementation does not support categorical variables for now. Other techniques are usually specialized in analyzing datasets that have only one type of variable.
- * Able to handle multi-output problems.
- * Uses a white-box model. If a given situation is observable in a model, the explanation for the condition is easily explained by boolean logic. By contrast, in a black box model (eg, in an artificial neural network), results may be more difficult to interpret.
- * Possible to validate a model using statistical tests. That makes it possible to account for the reliability of the model.
- * Performs well even if its assumptions are somewhat violated by the true model from which the data were generated.

• Disadvantages of Decision Trees

* Decision-Tree learners can create over-complex trees that do not generalise the data well!

This is called overfitting. Mechanisms such as pruning, setting the min. no. of samples required at a leaf node or setting the max depth of the tree are necessary to avoid this problem.

* Decision Trees can be unstable because small variations in the data might result in a completely different tree being generated. This problem is mitigated by using decision trees within an ensemble.

* Predictions of decision trees are neither smooth nor continuous, but piecewise constant approximations as seen in the above figure. Therefore, they are not good at extrapolation.

* The problem of learning an optimal decision tree is known to be NP-complete under several aspects of optimality and even for simple concepts. Consequently, practical decision-tree learning algorithms are based on heuristic algorithms such as the greedy algorithm where locally optimal decisions are made at each node. Such algorithms cannot guarantee to return the globally optimal decision tree. This can be mitigated by training multiple trees in an ensemble learner where the features and

with replacement

- Ensembles: Random Forest

Bagging Code Samples $\rightarrow$ Scikit-learn
Random Forest ensemble classifier technique explained.

• Decision Tree Complexity:

* Training time complexity = $O(\underbrace{n * \lg n}_{\text{no. of pts in } D_{\text{Train}}} * \underbrace{d}_{\text{Dimensionality of the data}})$

* When we have numerical features ^{D_{Train}} it was very computationally expensive $\because$ each value needed to be taken as a threshold. But, there are algorithmic efficient methods and hacks to handle numerical features efficiently.

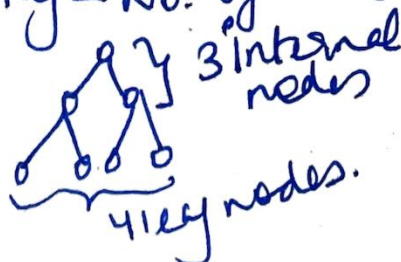
* So, $(n * \lg n)$ part is done for sorting the numerical features. n is number of features in numerical columns.

* At every stage of building your decision tree you have to evaluate each features to compute the information gain. So, if you have your dataset of large depth ' d ', decision trees may not be the best option $\because$ you have ' d ' here as your multiplier.

* So, at runtime, after training, the space at runtime/evaluation time, if given an x_q , if it is need to tell y_q , we need to store the decision tree and it is very easy using nested if-else. So, normally, after training, decision trees are converted into nested if-else conditions and it is very space efficient.

* So, the space complexity @ runtime is the no. of internal nodes + no. of leaf nodes.

Space complexity = No. of nodes.



* And if decision tree is not too deep / of reasonable depth. Then it is not very difficult.

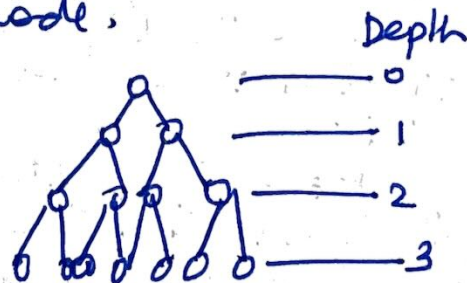
* We don't train decision tree of depth 5 or 10 not 100.

If there is massive dataset, we don't go too deep. Because, interpretability decreases and entire value affects a lot.

* Default value of depth around 3 is taken.

* Runtime Space Complexity = Reasonable
Time " = Given X_q to determine Y_q

we are going to make a decision for k decisions where k is the max depth of any leaf node.



* So, we will do utmost 3 comparison operators. So, depth of the tree is $O(d)$, which is $\text{depth}(d) = O(3)$.

* So, DT are good when we have large data, dimensions are low & low-latency requirement. $\therefore$ time complexity is $O(\text{depth})$.